

Data Structures and Algorithm Analysis – Lab Exercise 4

Submitted By: 25MCB1010, Patsamatla Bhaavya Ramita

Email: patsamatla.bhaavya2025@vitstudent.ac.in

Q1. **Problem Statement:** Emma is working on a navigation system that needs to find the distance between two nodes in a Binary Search Tree (BST). Implement a function to help Emma calculate the distance between two nodes.

Code:

```
#include <stdio.h>

#include <stdlib.h>

struct Node {
    int val;
    struct Node* left;
    struct Node* right;
};

struct Node* newNode(int val) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->val = val;
    temp->left = temp->right = NULL;
}

struct Node* insert(struct Node* root, int val) {
    if(root == NULL) {
        return newNode(val);
    }
    if(root->val > val) {
        root->left = insert(root->left, val);
    }
    else if(root->val < val) {
        root->right = insert(root->right, val);
    }
    return root;
}
```

```

struct Node* common(struct Node* root, int n1, int n2) {
    if(root == NULL) {
        return NULL;
    }
    if(n1 < root->val && n2 < root->val) {
        return common(root->left, n1, n2);
    }
    else if(n1 > root->val && n2 > root->val) {
        return common(root->right, n1, n2);
    }
    return root;
}

int distance(struct Node* root, int key) {
    if(root->val == key) {
        return 0;
    }
    if(key < root->val) {
        return 1 + distance(root->left, key);
    }
    else {
        return 1 + distance(root->right, key);
    }
}

int result(struct Node* root, int n1, int n2) {
    struct Node* lca = common(root, n1, n2);
    int d1 = distance(lca, n1);
    int d2 = distance(lca, n2);
    return d1 + d2;
}

int main() {
    int t;

    printf("Enter the number of test cases: ");

```

```

scanf("%d", &t);
for(int k=0; k<t; k++) {
    int n;

    int val;

    struct Node* root = NULL;

    printf("Enter the number of nodes: ");
    scanf("%d", &n);
    for(int i=0; i<n; i++) {
        val = 0;

        printf("Enter the node value: ");
        scanf("%d", &val);

        root = insert(root, val);
    }

    int node1, node2;

    printf("Enter the two nodes: ");
    scanf("%d", &node1);
    scanf("%d", &node2);

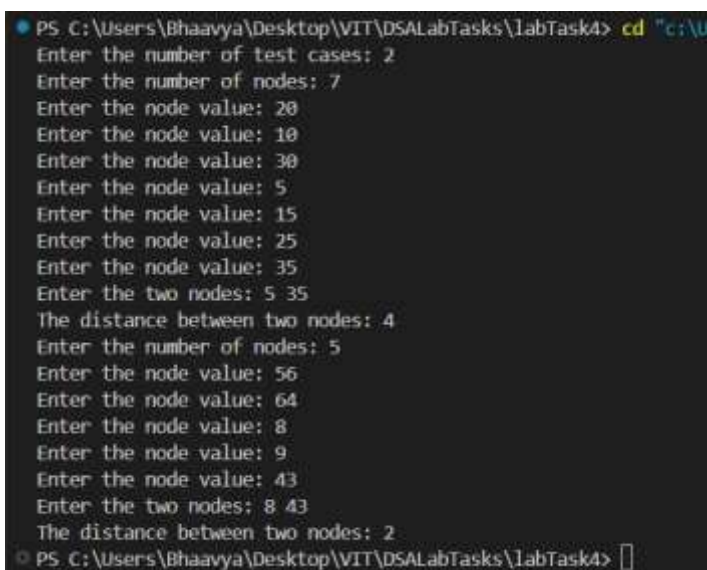
    int res = result(root, node1, node2);

    printf("The distance between two nodes: %d\n", res);
}

return 0; }

```

Output:



```

PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks\labTask4> cd "c:\Users\Bhaavya\Desktop\VIT\DSALabTasks\labTask4"
Enter the number of test cases: 2
Enter the number of nodes: 7
Enter the node value: 20
Enter the node value: 10
Enter the node value: 30
Enter the node value: 5
Enter the node value: 15
Enter the node value: 25
Enter the node value: 35
Enter the two nodes: 5 35
The distance between two nodes: 4
Enter the number of nodes: 5
Enter the node value: 56
Enter the node value: 64
Enter the node value: 8
Enter the node value: 9
Enter the node value: 43
Enter the two nodes: 8 43
The distance between two nodes: 2
PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks\labTask4>

```

Q2. Problem Statement: Jay is studying binary search trees (BSTs) as part of his computer science curriculum. He is particularly interested in understanding operations like insertion and deletion in a BST. Jay wants to write a program that takes a sequence of characters as input, constructs a BST using those characters, deletes the minimum element from the BST, and then outputs the elements of the tree in sorted order. Help Jay by designing a program that achieves this.

Code:

```
#include <stdio.h>

#include <stdlib.h>

struct Node {
    char val;
    struct Node* left;
    struct Node* right;
};

struct Node* newNode(int val) {
    struct Node* root = (struct Node*)malloc(sizeof(struct Node));
    root->val = val;
    root->left = root->right = NULL;
    return root;
}

struct Node* insert(struct Node* root, int val) {
    if(root == NULL) {
        root = newNode(val);
    }
    if(val < root->val) {
        root->left = insert(root->left, val);
    }
    else if(val > root->val) {
        root->right = insert(root->right, val);
    }
    return root;
}

struct Node* deleteMin(struct Node* root) {
```

```

if(root == NULL) {
    return NULL;
}

if(root->left == NULL) {
    struct Node* temp = root->right;
    free(root);
    return temp;
}

root->left = deleteMin(root->left);
return root;
}

void inorder(struct Node* root) {
    if(root != NULL) {
        inorder(root->left);
        printf("%c ", root->val);
        inorder(root->right);
    }
}

int main() {
    int test;

    printf("Enter the number of test cases: ");
    scanf("%d", &test);

    for(int t=0; t<test; t++) {
        int n;

        int val = 0;

        printf("Enter the number of elements: ");
        scanf("%d", &n);

        struct Node* root = NULL;

        printf("Enter the elements: ");

        for(int i=0; i<n; i++) {
            scanf(" %c", &val);

            root = insert(root, val);

```

```

    }

    root = deleteMin(root);

    printf("Inorder traversal after deleting the minimum element:\n");

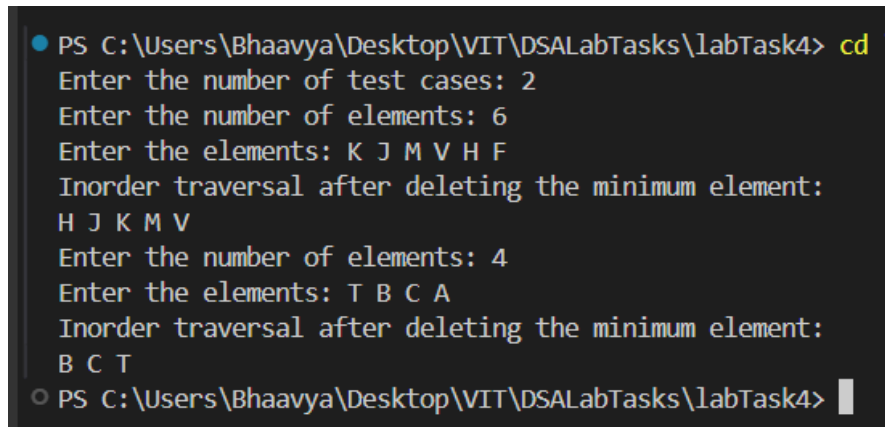
    inorder(root);

    printf("\n");
}

return 0; }

```

Output:



```

PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks\labTask4> cd ""
Enter the number of test cases: 2
Enter the number of elements: 6
Enter the elements: K J M V H F
Inorder traversal after deleting the minimum element:
H J K M V
Enter the number of elements: 4
Enter the elements: T B C A
Inorder traversal after deleting the minimum element:
B C T
PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks\labTask4>

```

Q3. **Problem Statement:** Rao wants to analyze the frequency of words in a collection of text. He plans to implement a program using a Binary Search Tree (BST) to efficiently store and manage the words along with their frequencies. The program should read words from the input until the word "end" is encountered, construct the BST, and then output the words in alphabetical order along with their frequencies. Additionally, Rao wants to find and display the most frequent word in the text.

Code:

```

#include <stdio.h>

#include <string.h>

#include <stdlib.h>

struct Node {

    char val[100];

    int count;

    struct Node* left;

    struct Node* right;

};

```

```

struct Node* newNode(char* val) {
    struct Node* root = (struct Node*)malloc(sizeof(struct Node));
    strcpy(root->val, val);
    root->count = 1;
    root->left = root->right = NULL;
    return root; }

struct Node* insert(struct Node* root, char* val) {
    if (root == NULL) {
        return newNode(val); }
    int cmp = strcmp(val, root->val);
    if (cmp == 0) {
        root->count += 1;
    } else if (cmp < 0) {
        root->left = insert(root->left, val);
    } else {
        root->right = insert(root->right, val); }
    return root; }

void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%s (%d times)\n", root->val, root->count);
        inorder(root->right);
    } }

void freq(struct Node* root, char* word, int* maxCount) {
    if (root != NULL) {
        freq(root->left, word, maxCount);
        if (root->count > *maxCount) {
            *maxCount = root->count;
            strcpy(word, root->val); }
        freq(root->right, word, maxCount);
    } }

int main() {

```

```

int test;

printf("Enter the number of test cases: ");

scanf("%d", &test);

for (int t = 0; t < test; t++) {

    struct Node* root = NULL;

    char val[100];

    printf("Enter words: ");

    while (scanf("%s", val)) {

        if (strcmp(val, "end") == 0) {

            break; }

        root = insert(root, val); }

    printf("\nWords in alphabetical order:\n");

    inorder(root);

    char res[100] = "";

    int maxCount = 0;

    freq(root, res, &maxCount);

    if (maxCount > 0) {

        printf("\nThe most frequent word: %s (%d times)\n", res, maxCount);

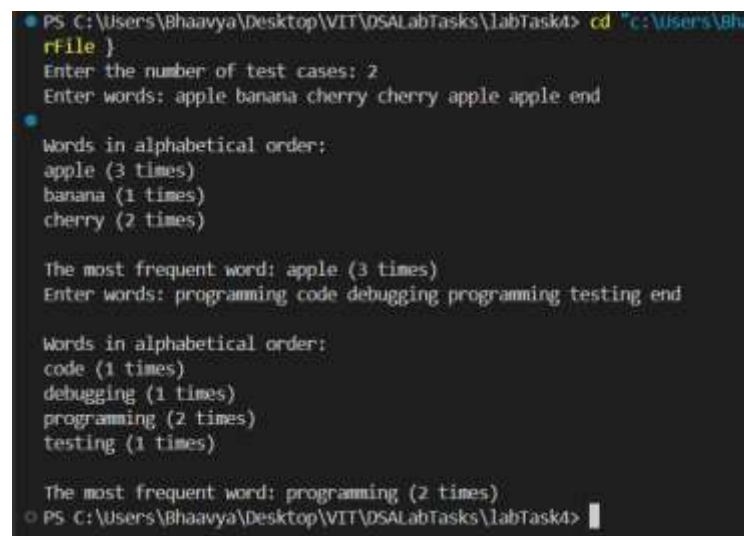
    }

}

return 0;
}

```

Output:



```

PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks\labTask4> cd "c:\Users\Bhaavya\Desktop\VIT\DSALabTasks\labTask4"
rfile }
Enter the number of test cases: 2
Enter words: apple banana cherry cherry apple apple end

Words in alphabetical order:
apple (3 times)
banana (1 times)
cherry (2 times)

The most frequent word: apple (3 times)
Enter words: programming code debugging programming testing end

Words in alphabetical order:
code (1 times)
debugging (1 times)
programming (2 times)
testing (1 times)

The most frequent word: programming (2 times)
PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks\labTask4>

```


Q4. Problem Statement: Emily is studying binary search trees (BST). She wants to write a program that inserts characters into a BST and then finds and prints the minimum and maximum values. Guide her with the program.

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int val;
    struct Node* left;
    struct Node* right;
};

struct Node* newNode(int val) {
    struct Node* root = (struct Node*)malloc(sizeof(struct Node));
    root->val = val;
    root->left = root->right = NULL;
    return root;
}

struct Node* insert(struct Node* root, int val) {
    if(root == NULL) {
        root = newNode(val);
    }
    else if(root->val < val) {
        root->right = insert(root->right, val);
    }
    else if(root->val > val) {
        root->left = insert(root->left, val);
    }
    return root;
}

char min(struct Node* root) {
    if(root == NULL) {
        return '\0';
    }
}
```

```

    }
    while(root->left != NULL) {
        return min(root->left);
    }
    return root->val;
}

char max(struct Node* root) {
    if(root == NULL) {
        return '\0';
    }
    while(root->right != NULL) {
        return max(root->right);
    }
    return root->val;
}

int main() {
    int test;

    printf("Enter the numbe rof test cases: ");
    scanf("%d", &test);
    for(int t=0; t<test; t++) {
        struct Node* root = NULL;

        int n;

        char val;

        printf("Enter the number of elements: ");
        scanf("%d", &n);
        for(int i=0; i<n; i++) {
            scanf(" %c", &val);
            root = insert(root, val);
        }

        printf("Minimum value: %c\n", min(root));
        printf("Maximum value: %c\n", max(root));
    }
}

```

```

    return 0;
}

```

Output:

```

PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks\labTask4> cd ..
Enter the number of test cases: 2
Enter the number of elements: 5
Z E W T Y
Minimum value: E
Maximum value: Z
Enter the number of elements: 7
S P O R T Y U
Minimum value: O
Maximum value: Y

```

Q5. **Problem Statement:** Jasmina needs a program that performs a postorder traversal on a binary tree constructed from an array of integers. The program should take an integer n as input, followed by n integers representing the elements of the array. It constructs a binary tree using the array elements in level order and then performs a postorder traversal on the constructed binary tree. Finally, it prints the sequence of nodes visited during the postorder traversal.

Code:

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int val;
    struct Node* left;
    struct Node* right;
};

struct Node* newNode(int val) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->val = val;
    node->left = node->right = NULL;
    return node;
}

struct Queue {
    struct Node* nodes[25];
    int front, rear;
};

void enqueue(struct Queue* q, struct Node* node) {
    q->nodes[q->rear++] = node;
}

```

```

struct Node* dequeue(struct Queue* q) {
    return q->nodes[q->front++];
}

int isEmpty(struct Queue* q) {
    return q->front == q->rear;
}

void postorder(struct Node* root) {
    if(root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->val);
    }
}

int main() {
    int test;
    scanf("%d", &test);

    for(int t=0; t<test; t++) {
        int n;
        scanf("%d", &n);
        int arr[25];
        for(int i=0; i<n; i++) {
            scanf("%d", &arr[i]);
        }

        if(n == 0) continue;

        struct Node* root = newNode(arr[0]);
        struct Queue q = {.front=0, .rear=0};
        enqueue(&q, root);

        int i = 1;
        while(i < n) {
            struct Node* parent = dequeue(&q);
            parent->left = newNode(arr[i++]);
            enqueue(&q, parent->left);
            if(i >= n) break;
            parent->right = newNode(arr[i++]);
            enqueue(&q, parent->right);
        }
        postorder(root);
        printf("\n");
    }

    return 0;
}

```

Output:

```
PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks\labTask4> cd
● 2
  6
  9 8 5 4 3 1
  4 3 8 1 5 9
  5
  1 3 5 7 9
  7 9 3 5 1
○ PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks\labTask4> 
```

Submitted By: 25MCB1010, Patsamatla Bhaavya Ramita